INTRODUCTION

INTRODUCTION

Breast cancer remains one of the most prevalent and life-threatening forms of cancer affecting women worldwide, and early detection plays a critical role in improving survival rates and treatment outcomes. Traditional diagnostic methods rely heavily on the manual interpretation of ultrasound images by radiologists, a process that can be time-consuming and subject to human error or variability in expertise. With the advancement of deep learning and computer vision techniques, it has become possible to assist medical professionals with automated, accurate, and faster diagnostic tools.

This project presents a web-based Breast Cancer Detection System that leverages a hybrid deep learning model combining VGG16 (a pre-trained Convolutional Neural Network) with a Random Forest classifier to analyze breast ultrasound images and classify them as Benign or Malignant. The system is built using the Django framework and is designed with a multi-role architecture comprising Administrator, Doctor, User (patient), and Guest modules, enabling secure registration, verification, and role-based access. Users can upload ultrasound images through the platform, and the trained model processes the image to generate an instant prediction, supporting doctors and patients in making informed healthcare decisions

Objective

The primary objective of this project is to develop an automated system capable of classifying breast ultrasound images as Benign or Malignant using a deep learning model that combines VGG16 with a Random Forest classifier, thereby assisting medical practitioners in the early detection of breast cancer. The project aims to design a secure, role-based web platform with distinct access levels for Administrator, Doctor, User, and Guest, ensuring proper data management and verification workflows, while enabling user and doctor registration through an administrator-controlled verification and approval process before granting system access. It also focuses on providing a simple and intuitive interface for uploading ultrasound images and receiving real-time prediction results. Additionally, the project seeks to maintain organized records of users, doctors, districts, places, and categories through an administrative dashboard, supporting effective system management. Ultimately, the goal is to provide a supportive diagnostic tool that contributes to faster, more reliable clinical decision-making and better patient outcomes.

ABOUT THE ORGANIZATION

1. ABOUT THE ORGANIZATION

Progressive Software Solutions & Training is a fast developing, well established Information Technology enterprise with highly talented and efficient staff meeting the needs of clients throughout the world. The company is so popular that the clients feed the company with repeat orders, having faith in the capabilities. The staff includes management and engineering personnel, programmers, web developers, graphic designer, business analysts, technical writers etc. apart from the regular commercial and administrative hands. The motto of this reliable software provider is 'DO BEST; GET BEST' Progressive Software Solutions & Training committed to provide what they have promised. Progressive Software Solutions & Training strives for the satisfaction of their clients and dedicates themselves for the better services to their customers.

Progressive Software Solutions & Training are famous in providing high quality, cost cutting, requirement oriented and value added web and software solutions with remarkable cyber services. Progressive Software Solutions & Training are specialist in developing ever rewarding internet existence for ambitious companies which are in the hangover of swift moving internet facilities. Their aim is to grow with such organizations in a very fruitful manner. Their main concentration is to collect business oriented technological know-how and utilize them for the wellbeing of business concerns.

Progressive Software Solutions & Training experts in various technologies are committed to deliver effective and creative service to customers.

Services

- Software Development
- Web Development
- SEO Services (Search Engine Optimization)

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The existing system of police is a traditional, manual, and reactive approach to law enforcement. Here police officers manually write the complaints and the communication between officers, stakeholders etc are limited. The reactive is large and it has limited channels for public to report incidents. In this time eventually there are some kinds of websites and applications are there.

Disadvantage

- Time consuming

2.2 PROPOSED SYSTEM

Here the proposed system is make it computerized and help public to interact with police officers. It is a comprehensive, integrated solution designed to enhance public safety, prevent and investigate crimes, and provide emergency response services. This help the public and they can share their problem or even an incident to the authorities. Basically the project describes how to manage good performance and better services for user. Here we provide more security and privacy compare to existing system.

2.3 SOFTWARE REQUIREMENTS SPECIFICATION HARDWARE SPECIFICATION

The use of sufficient hardware is very important for the proper working of any software. Following are the hardware required by the system.

Processor	Intel Core i5
Processor Speed	1.5 GHz or higher
System Bus	32 Bit
Memory	1 GB RAM or higher
Hard Disk	40 GB or higher
Monitor	15.6” LCD Monitor
Keyboard	104 Keys

SOFTWARE SPECIFICATION

Software is a part of a computer system that consists of encoded information or computer instructions, in contrast to the physical hardware from which the system is built. Following are the software required by the system.

Operating System	Windows 11
IDE	Visual Studio code
Front End	PYTHON DJANGO, HTML 5
Back End	SQL3
Web Server	APACHE
Browser	Internet Explorer, Mozilla Firefox, Chrome

2.4 FEASIBILITY STUDY

The main objective of this study is to determine whether the proposed system is feasible or mainly there are three types of feasibility study to which the proposed system is subjected to as described below:

- Economic feasibility
- Technical feasibility
- Behavioral feasibility

The proposed system must be evaluated from a technical viewpoint first, and if technically feasible then their impact on organization must be accessed. If compatible, the behavioural system can be devised. Then those must be tested for economic feasibility.

ECONOMIC FEASIBILITY

Economic analysis is used for evaluating the cost effectiveness of a proposed system. Economically saying, our application is never cost related. It is easy to implement, and the user can conveniently use our application without any kind of exceptions or errors. Our application is very simple to interact is economically feasible.

TECHNICAL FEASIBILITY

The technical requirements of our system are highly affordable. There is no any difficulty in migrating from the existing to our proposed system. It is platform independent the application will run smoothly on any kernel without causing any trouble to end user.

BEHAVIORAL FEASIBILITY

The system is intended for personal uses. Since the software is simple and provides a good Graphical User Interface, it provides high level user.

2.5DATA FLOW DIAGRAM

To start the system design, something analogue to the architecture blue print as a starting point to design is required. It is a way to focus on functions rather than physical implementation. One such tool is a DFD.

Structured analysis is a set of techniques and graphical tools that help the analyst to develop a new kind of system specification that are easily understandable to the user. DFD's show the major decompositions of the system functions and their interfaces. The DFD is graphic and presents a picture of what is being specified and is conceptually easy to understand presentation of the application.

One important feature of DFD's is that it is logical rather than physical. The elements of the system do not depend on vendor or hardware. They specify in precise, concise manner the working of the system and how it hangs together.

DFD is the graphic representation of data movement process, and files used in support of an information system. There are several rules of thumb used in drawing DFDs.

- Process should be named and numbered for easy references.
- The direction of flow is from top to bottom and from left to right.
- When a process is imported in the lower levels details, they must be numbered.
- Process and data flow names have the first letter of each word must be a capital letter.

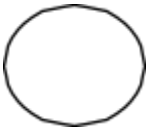
The symbol used in DFD are shown be



Source or destination of data



Data flow

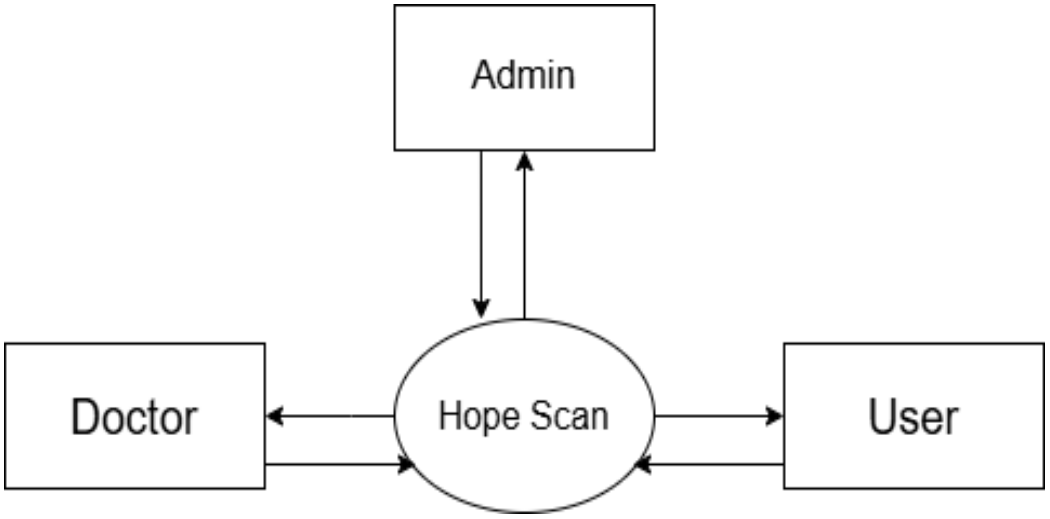


Process that transform data flow



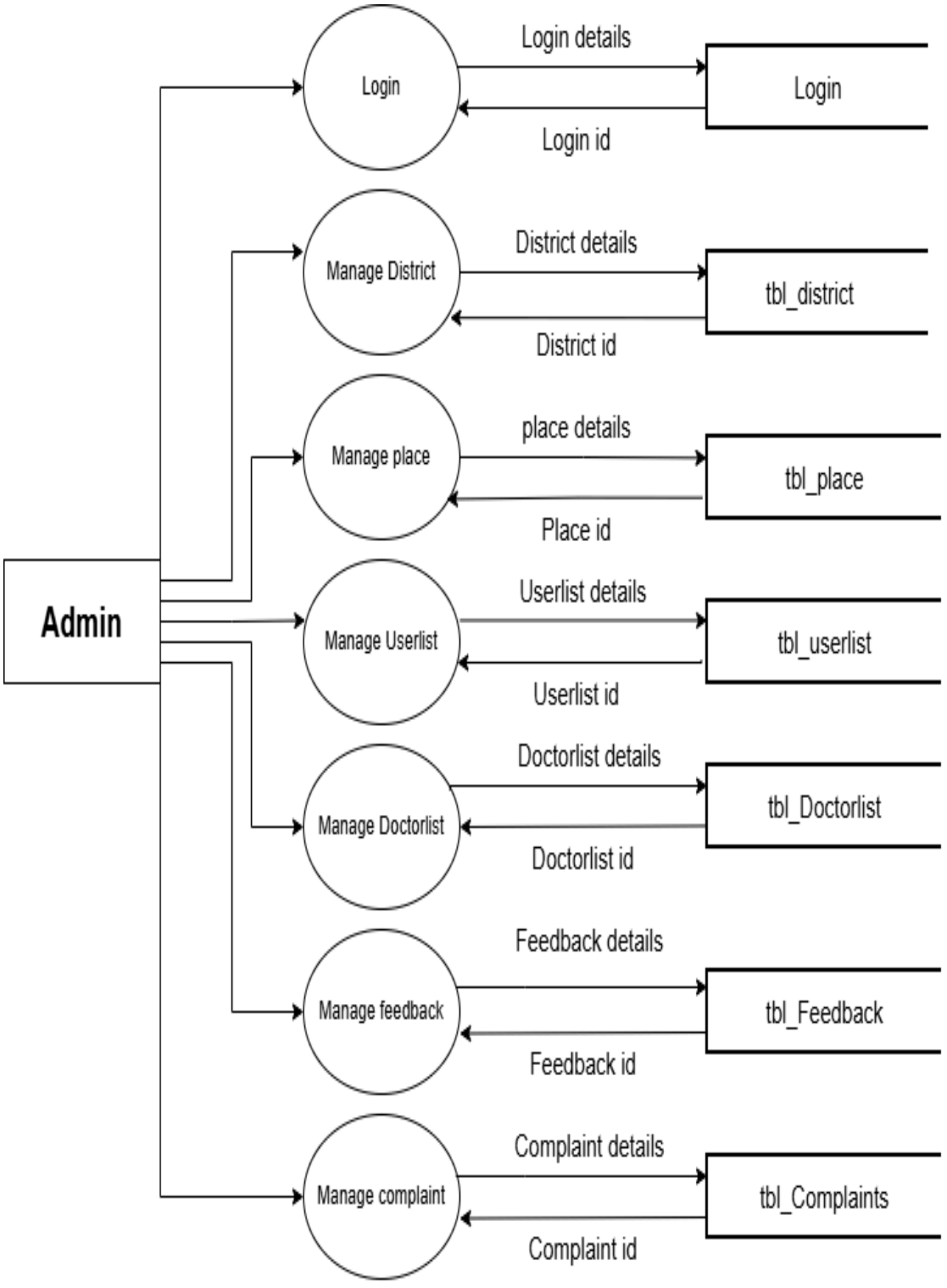
Data store

CONTEXT LEVEL

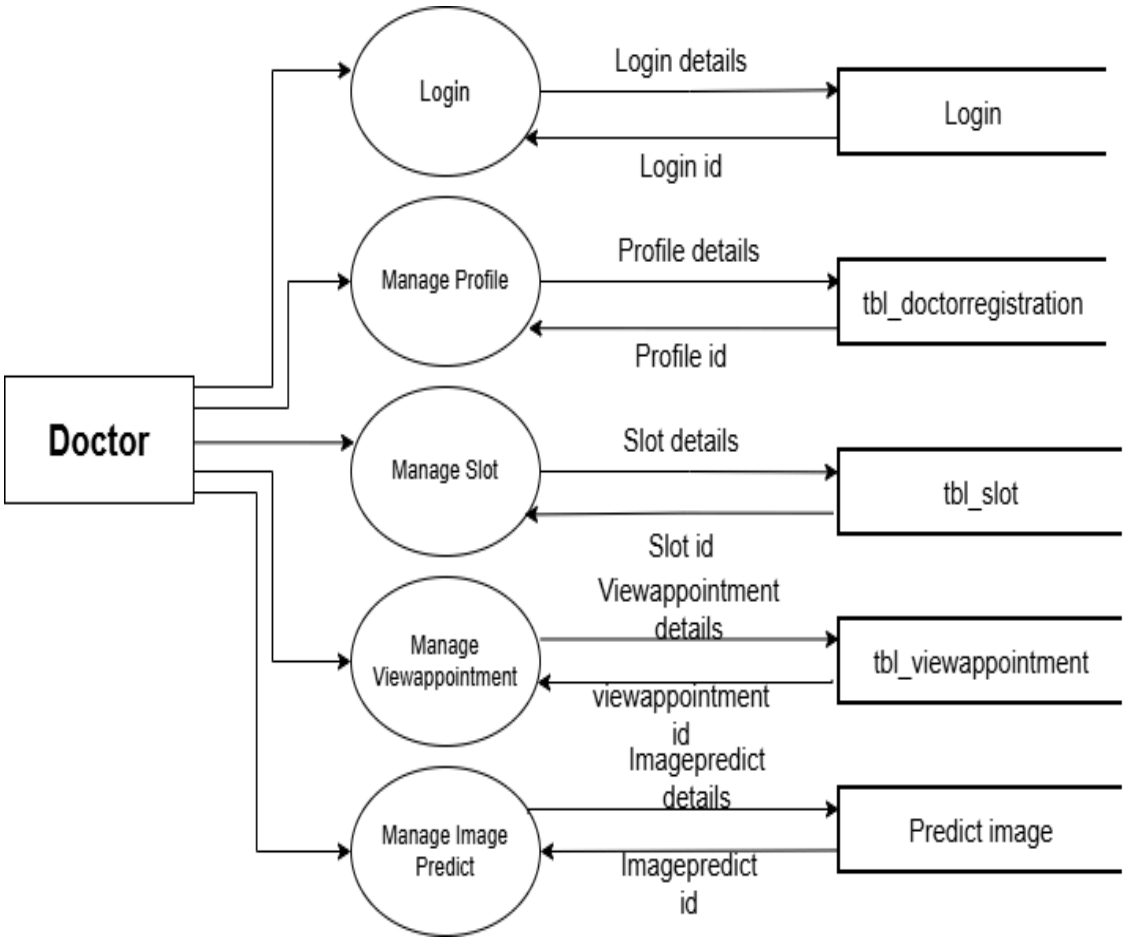


-

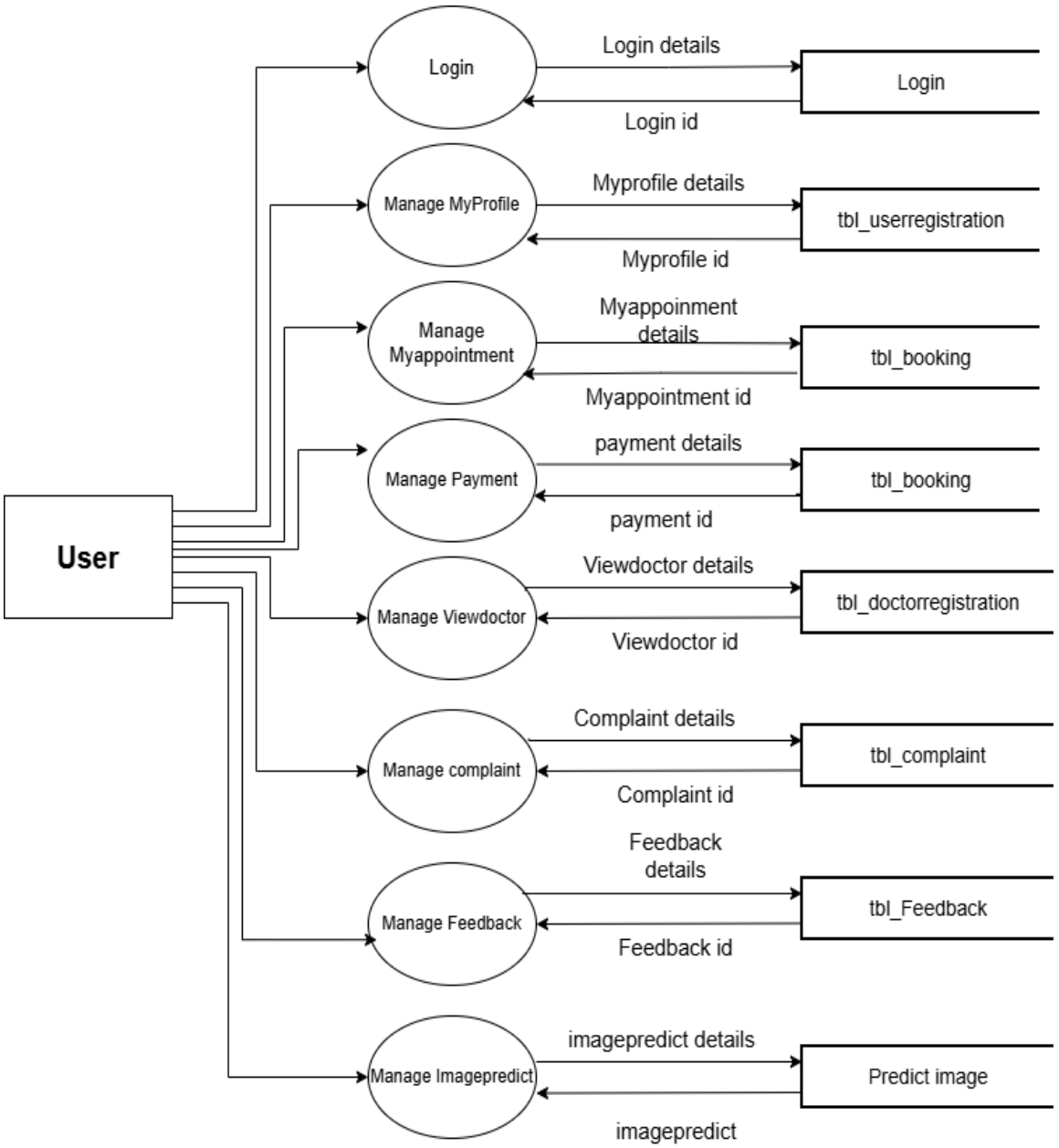
LEVEL 1 FOR ADMIN



LEVEL 1 FOR DOCTOR



LEVEL 1 FOR USE



SYSTEM DESIGN

3. SYSTEM DESIGN

System design is the creative act of invention developing new inputs, a database offline files, method, procedures and output for processing business data to meet organization objectives. The design phase focuses on the detailed implementation of the system recommended in the feasibility study. The design phase is a transition from user-oriented document to a document oriented to the programmers or database personnel. Characteristics of a well-designed system are:

- Accessibility
- Decision making ability
- Economy
- Flexibility
- Reliability
- Simplicity

The design will determine the success of the system. System design is based on the information gathered during system analysis. System design goes through two phases of development.

- Logical design- DFD shows the logical flow of a system and defines the boundaries of the system. For the candidate the system it describes the input, outputs, databases and procedures-all in a format that meets the user's requirements.
- Physical design - This produces the working system by defining the design specification that tells programmers exactly what the candidate system must do.

3.1 INPUT DESIGN

Input design is the link that ties the information system into the world of its users. The input design involves determining what the inputs are, how the data should be performed, how to validate data, how minimize data entry and how to provide a multi-user facility inaccurate.

Input data are the most common cause of error in data processing. Errors entered by data entry operators can be controlled by input design. Input design is the process of converting user.

Originated inputs to a computer-based format. Input data are collected and organized into groups of similar data. Once identified, appropriate input media are selected for processing. All the input data validated in the order and if any data violates any condition, the user is warned by a message. If the data satisfies all the conditions, then it is transferred to the appropriate tables in the database.

3.2 OUTPUT DESIGN

Output design involves specifying how production of on-screen reports and page based reports will occur. Output may occur to database or file for storing information entered or also for use by other systems. Computer output is the most important and direct source of information to the user. Output design is a very important phase because the output needs to be in an attractive manner. Efficient, intelligible output design improves the system relationship with the user and help in decision making. A major form of the output is the hard copy from the printer and screen reports. Printouts are designed around the output requirements of the user. Allowing the user to view to the sample screen is important because the user is ultimate judge of the quality of output. The output model of this system is the user-friendly window. These user-friendly windows are meant for the purpose of easy view of the stored information.

3.3 TABLE DESIGN

A database is a collection of data. Database design refers to the design of the tables used to store data. The database involves name of records, data item with its name, type and size. In the design of the database program first we must thoroughly look into the requirements of the program for the design of database. Then we must design how much tables are required in the database. Thereafter as per the requirement of the end users we can decide which fields that must be in this table. As per a general rule a provision must be taken in the design for the future enhancement of the program. Some of important tables are:

TABLE STRUCTURE

1.Table name: tbl_adminregistration
Table description: To store the details of admin

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
adminregistration_id	int(10)	Primary key	Admin id
adminregistration_name	varchar(30)	Not null	Name of the admin
adminregistration_password	varchar(30)	Not null	Password of the admin
adminregistration_email	varchar(50)	Not null	Email of the admin

2. Table name: tbl_district
Table description: To store the details of district

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
district_id	int(10)	Primary key	District id
district_name	varchar(30)	Not null	Name of the district

3. Table name: tbl_place

Table description: To store the details of place

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
place_id	int(10)	Primary key	Place id
place_name	varchar(30)	Not null	Name of the place
district_id	int(10)	Foreign key (tbl_district)	District id

4.Table name:tbl_category

Table description: To store the details of category of the complaint

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
category_id	int(10)	Primary key	Category id
category_name	varchar(30)	Not null	Name of Category

t

6.Table name: tbl_subcategory

Table description:To store the details of Subcategory of the complaint

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
subcategory_id	int(10)	Primary key	subcategory_id
subcategory_name	varchar(300)	Not null	Subcategory Name
category_id	int(10)	Foreign key (tbl_category)	Category id

6.Table name: tbl_slot

Table description: To manage Doctor slot

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
slot_id	int(11)	Primary key	Slot id
slot_date	Date	Not null	Name of Slot date
slot_fromtime	int(11)	Not null	Name of Slot from time
slot_totime	int(11)	Not null	Name of Slot totime
slot_count	int(11)	Not null	Slot count
doctor_id	varchar(25)	Foreign key (tbl_doctorregistration)	Password of policestation

7.Table name: tbl_userregistration

Table description: To store the details of the user

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
userregistration_id	int(11)	Primary key	Event Id
userregistration_name	varchar(25)	Not null	Name of user
userregistration_email	varchar(25)	Not null	Email of user
userregistration_password	varchar(25)	Not null	Password of user
userregistration_gender	varchar(20)	Not null	Gender of user
userregistration_contact	varchar(300)	Not null	Contact of user
userregistration_address	varchar(300)	Not null	Address of the user
userregistration_photo	varchar(300)	Not null	Photo of user
user _status	varchar(30)	Not null	Status of user
place_id	int(10)	Foreign key	Place id
userregistration_dateofbirth	Date	Not null	Date of Birth of user

8.Table name: tbl_doctorregistration

Table description: To store children details

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
doctorregistration_id	int(10)	Primary key	Doctor registration Id
doctorregistration_name	varchar(25)	Not null	Doctor name
doctorregistration_email	varchar(25)	Not null	Doctor email
doctorregistration_password	varchar(25)	Not null	Password Of Doctor registration
doctorregistration_address	varchar(100)	Not null	Address of Doctor
doctorregistration_contact	int(25)	Not null	Contact of Doctor
doctorregistration_photo	varchar(25)	Not null	Photo of doctor
doctorregistration_proof	varchar(25)	Not null	Id proof of doctor
doctorregistration_gender	varchar(15)	Not null	Gender of doctor
doctor_status	int(10)	Not null	Status of doctor
doctorregistration_place_id	int(10)	Foreign key	Place id
doctorregistration_exp	int(10)	Not null	Experience of doctor

9.Table name: tbl_booking"

Table description: To store Booking details of user

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
booking_id	int(10)	Primary key	Booking Id
booking_date	Date	Not null	Booking date
booking_time	time	Not null	Time of booking
booking_amount	int(10)	Not null	Booking amount
booking_status	int(10)	Not null	Status of booking

10.Table name: tbl_complaint

Table description: User complaints

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
complaint_id	int(10)	Primary key	Complaint Id
complaint_file	varchar(250)	Not null	file of the complaint
complaint_content	varchar(250)	Not null	Content of the complaint
complaint_date	date	Not null	Date of the complaint
complaint_status	varchar(25)	Not null	Status of the complaint
complaint_reply	varchar(25)	Not null	Reply of complaint

11.Table name: tbl_feedback

Table description: Feedback for our website

Field Name	Data Type	Constraints (Primary key, foreign key, not null, null)	Field Description
feedback_id	int(10)	Primary key	Feedback Id
feedback_content	varchar(300)	Not null	Content of feedback
feedback_date	date	Not null	Date of the feedback
user_id	int(10)	Foreign key	User Id

3.4 PROCESS DEIGN

There are 4 modules in the system.

1. Admin
2. Police Station
3. User
4. Children

1. ADMIN

- Add district

  Administrator has the duty to add district

- Add place

  Administrator has the duty to add place

- Add doctor

  Administrator has the duty to add doctor

- Add user

  Administrator has the duty to accept and reject

- Manage complaint

  Administrator has the duty to manage the complaint

2. DOCTOR

- Add slot

  Doctor has the duty to add slot

- Manage payment

  Doctorhas the duty to add payment.

3. USER

- User can login

  All user can login.

- Add appointment

  User can make appointment

- Add feedback

  User can add feedback

- Add complaint

  User can add website complaints.

SYSTEM TESTING AND IMPLEMENTATION

4. SYSTEM TESTING & IMPLEMENTATION

Testing is the process of examining the software to compare the actual behaviour with that of the expected behaviour. The major goal of software testing is to demonstrate that faults are not present. In order to achieve this goal the tester executes the program with the intent of finding errors. Though testing cannot show absence of errors but by not showing their presence it is considered that these are not present.

System testing is defined as the process by which one detects the defects in the software. Any software development organization or team has to perform several processes. Software testing is one among them. It is the final opportunity of any programmer to detect and rectify any defects that may have appeared during the software development stage. Testing is a process of testing a program with the explicit intention of finding errors that makes the program fail. In short system testing and quality assurance is a review in software products and related documentation for completion, correctness, reliability and maintainability.

System testing is the first stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is vital to the success of the system. System testing makes a logical assumption that if all the parts of the system are correct and the goal will be successfully achieved. A series of testing are performed for the proposed system before the proposed system is ready for user acceptance testing.

The testing steps are:

- Unit Testing
- Integration Testing
- Validation Testing
- Output Testing
- Acceptance Testing

System Testing provides the file assurance that software once validated must be combined with all other system elements. System testing verifies whether all elements have been combined properly and that overall system function and performance is achieved. After the integration of modules, the validation test was carried out over the system. It was found that all the modules work well together and meet the overall system function and performance.

- **UNIT TESTING**

Unit testing is carried out screen-wise, each screen being identified as an object. Attention is diverted to individual modules, independently to one another to locate errors. This has enabled the detection of errors in coding and logic. Various test cases are prepared. For each module these test cases are implemented and it is checked whether the module is executed as per the requirements and outputs the desired result. In this test each service input and output parameters are checked. In unit testing,

- Module interface was tested to ensure that information properly flows into and out of the program under test.
- Boundary condition was tested to ensure that module operates properly at boundaries established to limit or restrict processing.
- All independent paths through the control structures were executed to ensure that all statements in the modules have been executed at least once.
- Error handling paths were also tested.

- **INTEGRATION TESTING**

Integration testing is a systematic technique for constructing the program structure at the same time conducting tests to uncover errors associated with interfacing. Unit tested module were taken and a single program structure was built that has been

dictated by the design. Incremental integration has been adopted here.

Integration testing is a systematic technique for constructing the program structure at the same time conducting tests to uncover errors associated with interfacing. Unit test module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here.

Integration testing is a systematic technique for constructing the program structure at the same time conducting tests to uncover errors associated with interfacing. Unit tested module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here. The modules are tested separately for accuracy and modules are integrated together using bottom-up integration i.e., by integrating from moving from bottom to the top. The system is checked and errors found during integration are rectified. In this testing, all the individual modules were combined and the module wise shifting was verified to be alright. The entire software was developed and tested in small segments, where errors were easy to locate and rectify. Program builds (group of modules) were constructed corresponding to the successful testing of user interaction, data manipulation analysis, and display processing and database management.

- **VALIDATION TESTING**

Validation testing is done to ensure complete assembly of the error free software. Validation can be termed successful only if it functions in manner. Reasonably expected by the customer under validation is alpha and beta testing. Alpha testing is where the end user tests the system rather than the developer, but in a controlled environment. The software is used on a natural setting with the developer monitoring the user using the system. The developer records the errors and usage problems encountered by the user. Alpha testing is simulated or actual operational testing by potential users/customers or an independent test team at the developers' site. Alpha testing is often employed for off-the- shelf software as a form of internal acceptance testing, before the software goes to beta testing.

Beta testing comes after alpha testing. Versions of the software, known as beta versions, are released to a limited audience outside of the programming team. The software is released to groups of people so that further testing can ensure the product has few faults or bugs. Sometimes, beta versions are made available to the open public to increase the feedback field to a maximal number of future users.The user person conducts beta testing at one more site. The developer is not present during these tests. Hence, beta test can be said as the live application of software on an environment that cannot be controlled by the developer. The sales person takes down the problems encountered during beta testing and reports to the developer at regular intervals. The developer makes suitable modifications to the software henceforth. The client-side validation is done in this testing phase. It is checked whether the data passed to each service is valid or not. Entering incorrect values in the field of email id, passwords, mobile number and name are checked in this project. If any, those values are discarded and a message will be appeared on the screen. Like this the errors are rectified.

- **OUTPUT TESTING**

After performing the validation testing the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in a specific format. Asking the users about the format required by them, testing the output generated are considered into two ways. One is onscreen and another is printed format. The output format on the screen was found to be correct as the format was designed in the system design phase according to the user needs. For the hard copy also, the output comes out as specified requirement by the user. Hence output testing does not result in any correction in the system.

"CYBROX" is developed based on the user choice. It is user friendly. The output format is very clear to user.

- **ACCEPTANCE TESTING**

Acceptance testing generally involves running a suite of tests on the completed system. Each individual test, known as a case, exercises a particular operating condition of the user's environment or feature of the system, and will result in a pass or fail, or Boolean, outcome.

Acceptance testing is the user testing. I.e., whether the user is accepted or rejected the system. "CYBROX" is developed for peoples. Its screens are created as the base of user's choice. CYBROX it helps to restore and preserve rivers through community-led cleanups. The software passed the acceptance test correctly. Screens are created as the base of user's choice.

- **DEBUGGING**

Debugging is the process of detecting and removing of existing and potential errors (also called as 'bugs') in a software code that can cause it to behave unexpectedly or crash. To prevent incorrect operation of a software or system, debugging is used to find and resolve bugs or defects. When various subsystems or modules are tightly coupled, debugging becomes harder as any change in one module may cause more bugs to appear in another. Sometimes it takes more time to debug a program than to code it.

To debug a program, user has to start with a problem, isolate the source code of the problem, and then fix it. A user of a program must know how to fix the problem as knowledge about problem analysis is expected. When the bug is fixed, then the software is ready to use. Debugging tools (called debuggers) are used to identify coding errors at various development stages. They are used to reproduce the conditions in which error has occurred, then examine the program state at that time and locate the cause. Programmers can trace the program execution step-by-step by evaluating the value of variables and stop the execution wherever required to get the value of variables or reset the program variables. Some programming language packages provide a debugger for checking the code for errors while it is being written at run time.

Here is the debugging process:

1. Reproduce the problem.
2. Describe the bug. Try to get as much input from the user to get the exact reason.
3. Capture the program snapshot when the bug appears. Try to get all the variable values and states of the program at that time.
4. Analyse the snapshot based on the state and action. Based on that try to find the cause of the bug.
5. Fix the existing bug, but also check that any new bug does not occur.

IMPLEMENTATION

The implementation is the final state and it is an important phase. It involves the individual programming; system testing, user training and the operational running of developed proposed system that constitutes the application subsystems. A major task of preparing for implementation is education of users, which should really have been taken place much earlier in the project when they were being involved in the investigation and design work. During the implementation phase system actually takes physical shape. In order to develop a system implemented planning is very essential.

The implementation phase of the software development is concerned with translating design specification into source code. The user tests the developed system and changes are made according to their needs. Our system has been successfully implemented. Before implementation several tests have been conducted to ensure that no errors are encountered during the operation. The implementation phase ends with an evaluation of the system after placing into the operation for a period of time.

The process of putting the developed system in actual use is called system implementation. This includes all those activities that take place to convert from old system to new system. The system can be implemented only after testing is done and is found to be working to specifications. The implementation stage is a systems project in its own right. The implementation stage involves following tasks:

IMPLIMENTATION PLANS:

- Test system with sample data
- Detection and correction of errors
- Make necessary changes in the system
- Check with the existing system
- Installation of hardware and software utilities
- Training and involvement of user personnel.

SECURITY TECHNOLOGIES AND POLICIES

.

5. SECURITY TECHNOLOGIES AND POLICIES

SYSTEM SECURITY

Threats to System Security

Any act or object that poses a danger to computer assets is known as threats. Management must be aware of the various kinds of threats facing the organization. By examining each threat category, the management can effectively protect the assets through policy, training and technology.

A threat can be either "intentional" (i.e. hacking: an individual cracker or a criminal organization) or "accidental" (e.g. the possibility of a computer malfunctioning, or the possibility of a natural disaster such as an earthquake, a fire, or a tornado) or otherwise a circumstance, capability, action, or event. There are several types of threats.

- Attempt to access websites and modify or destroy its contents
- Send malicious programs such as viruses, worms and Trojans to a web server by a browser
- Earthquake
- Excess electric supply
- Natural disasters.

SECURITY TECHNOLOGIES AND POLICIES

Maintenance involves the software industry captive, typing up system resources, it means restoring something to its original condition. Maintenance follows conversion to the extend. That change is necessary to maintain satisfactory operations relative to changes in the user’s environment. Maintenance often includes minor enhancements or corrections to problems that surface in the system's operation. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the so its software. Any system developed should be secured and

protected against possible hazards. Security measures are provided to prevent unauthorized access of the database at various levels. An uninterrupted power supply should be so that the power failure or voltage fluctuations will not erase the data in the files. Password protection and simple procedure to prevent the unauthorized access are provided to the users. The system allows the user to enter the system only through proper username and password.

SYSTEM SECURITY MEASURES

Security for e-commerce means complying with a number of key requirements, while making sure that online security measures are adopted. Having the right security measures in place is essential to maintain customer trust and confidence, and to ensure customer retention and loyalty.

To protect the confidential data from intruders, there are several security measures available today.

1. Passwords

The most inexpensive and perhaps the easiest method of security is achieved through password protection. A password is a secret word or string of characters that is used for user authentication to prove identity, or for access approval to gain access to a resource. Now passwords are commonly used by people to login to a computer system, access ATMs, etc. A password is also known as a passkey. It serves as a security measures against unauthorized access to data. Password security adds no cost to the network and is potentially a useful security measure.

MAINTENANCE

6. MAINTENANCE

Maintenance is the process of modifying a software system or component a after its delivery in order to correct faults, improve the performance and other attributes or to adapt to the changed environment. In **"HOPE SCAN"** maintenance covers a wide range of activities including correcting the coding and design errors, updating the documentation and test data and upgrading the user support. Maintenance involves the software industry captive, typing up system resources. It means restoring something to its original condition. Maintenance follows conversion to the extend. The change is necessary to maintain satisfactory operations relative to changes in the user's environment. Maintenance often includes minor enhancements or corrections to problems that surface in the system's operations. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the software. Hardware also requires periodic maintenance to keep the system in to its standards.

The future updates and maintenance can be done in this system even if it is delivered to the user. By modifying system after it has been put into use. Modifies the existing components and adds new components to the system. Maintenance is not only bug fixing but also adapting the software to changing requirements, changing environment etc.

Software maintenance is a very broad activity that includes error correction, enhancements of capabilities, deletion of obsolete capabilities, and optimization. Because change is inevitable, mechanisms must be developed for evaluating, controlling and making modifications. So any work done to change the software after it is in operation is considered to be maintenance work. The purpose is to preserve the value of software over time. The value can be enhanced by expanding the customer base, meeting additional requirements, becoming easier to use, more efficient and employing newer technology.

The categories of maintenance are:

- Corrective Maintenance
- Adaptive Maintenance
- Perfective Maintenance

CORRECTIVE MAINTENANCE

This refers to modifications initiated by defects in the software. A defect can result from design errors, logic errors and coding errors. Design errors occur when changes made to software are incorrect, incomplete, wrongly communicated or the change request is misunderstood. Logic errors result from invalid tests and conclusions, incorrect implementation of design specifications, faulty logic flow or incomplete test data. Coding errors are caused by implementation of detailed logic design and incorrect use of the source code logic. Defects are also caused by data processing errors and system performance errors.

In the event of system failure due to an error, actions are taken to restore the operation of the software system. Due to pressure from management, maintenance personnel sometimes resort to emergency fixes known as “patching”. The adhoc nature of this approach often gives rise to a range of problems that include increased program complexity and unforeseen ripple effects. Unforeseen ripple effects imply that a change to one part of a program may affect other sections in an unpredictable manner, thereby leading to distortion in the logic of the system. This is often due to lack of time to carry out through “impact analysis” before effecting the change.

ADAPTIVE MAINTENANCE

It includes modifying the software to match changes in the ever-changing environment. The term environment in this context refers to the totality of all conditions and influences which act from outside upon the software. A change to the whole or part of this environment will require a corresponding modification of the software.

Thus, this type of maintenance includes any work initiated as consequences of moving the software to a different hardware or software platform compiler, operating system or new processor. Any change in the government policy can have far-reaching ramifications on the software.

PERFECTIVE MAINTENANCE

It means improving processing efficiency or performance, or restricting the software to improve changeability. When the software becomes useful, the user tends to experiment with new cases beyond the scope for which it was initially developed. Expansion in requirements can take the form of enhancement of existing system functionality or improvement in computational efficiency.

Hence, perfective maintenance refers to enhancements: making the process better, faster, smaller, better documented, cleaner structured, with more functions or reports.

CONCLUSION

7. CONCLUSION

7.1 Scope for Future Enhancement

The entire project has been developed and deployed as per the requirements stated by the user, it is found to be bug free as per the testing standards that are implemented. Any specification- untraced errors will be concentrated in the coming versions, which are planned of be developed soon. The system at present does not take care of lower-level check constraints in accessing the file types in distributed environments, which is to be considered in the future up gradations. Here social media integration are there to enhance community engagement, improve response times, and increase transparency. We can use a virtual town hall for public discussion can be an effective way to engage citizens, foster transparency, and encourage participation. In this system we can apply AI and other functions in future. Real time tracking of children's and women's can be possible by AI integrations. We can Deploy drones for aerial surveillance, search and rescue, and crime scene investigation. Utilize advanced biometric technologies like facial recognition, fingerprint scanning, and iris scanning for security. Leverage cloud-based storage and analytics for efficient data management. We can convert this website to mobile apps for citizens to report crimes, submit tips, and access police services. Implement advanced threat detection systems to protect police data and systems. Provide mental health support and resources for officers. These enhancements can help police departments improve public safety, reduce crime, and build trust with the community.

The future updates and maintenance can be done in this system even if it is delivered to the user. By modifying a system after it has been put into use, modifies the existing components and adds new components to the system. Maintenance is not only bug fixing but also adapting the software to changing requirements, changing environment etc.

BIBLIOGRAPHY

8.BIBLIOGRAPHY

[1] Award, Elias M. ”System Analysis and Design”, Galgotia Publications Pvt Ltd,Year:2002,Edition 2.

[2] Silberschatz, Abraham, F, Henry and “Database System Concepts”, McGraw-Hill Education,Year:1986,Edition 6

[3] Aggarwal, K K, and Singh,Yogesh “Software Engineering” ,New Age International,Year:2005,Edition

REFERENCES

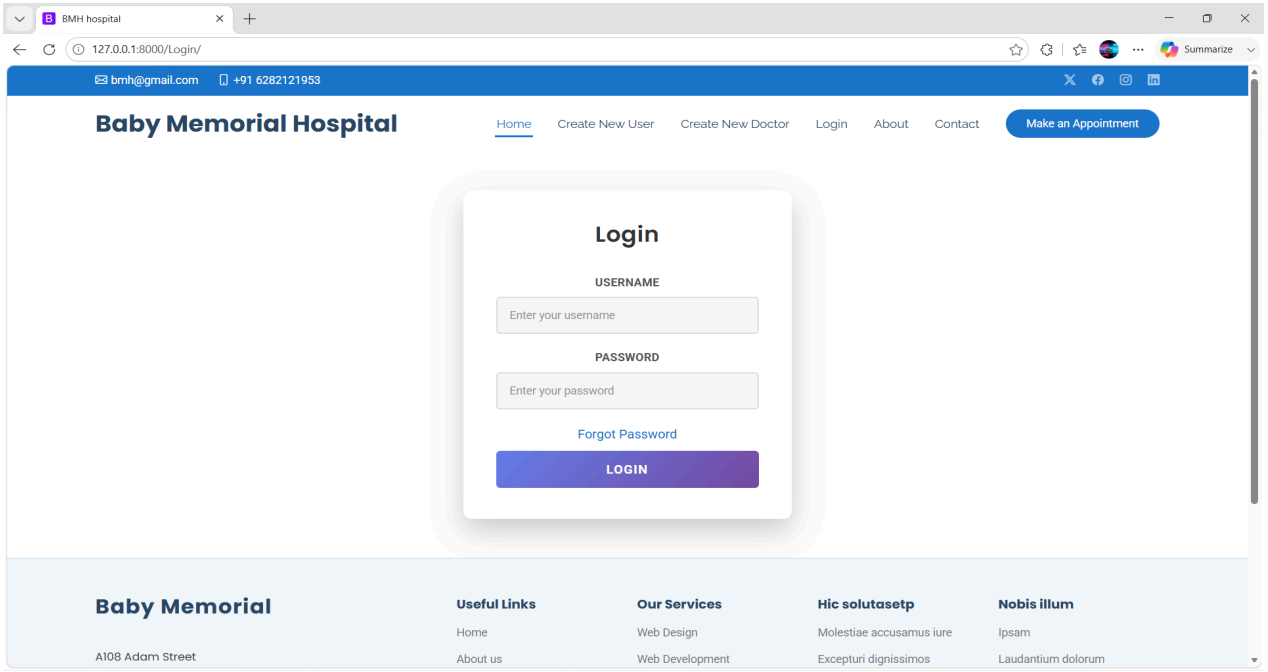
1. www.google.com
2. www.wikipedia.com
3. www.php.net
4. www.freecodecamp.org
5. www.w3schools.com
6. https://stacksoverflow.com

APPENDIX

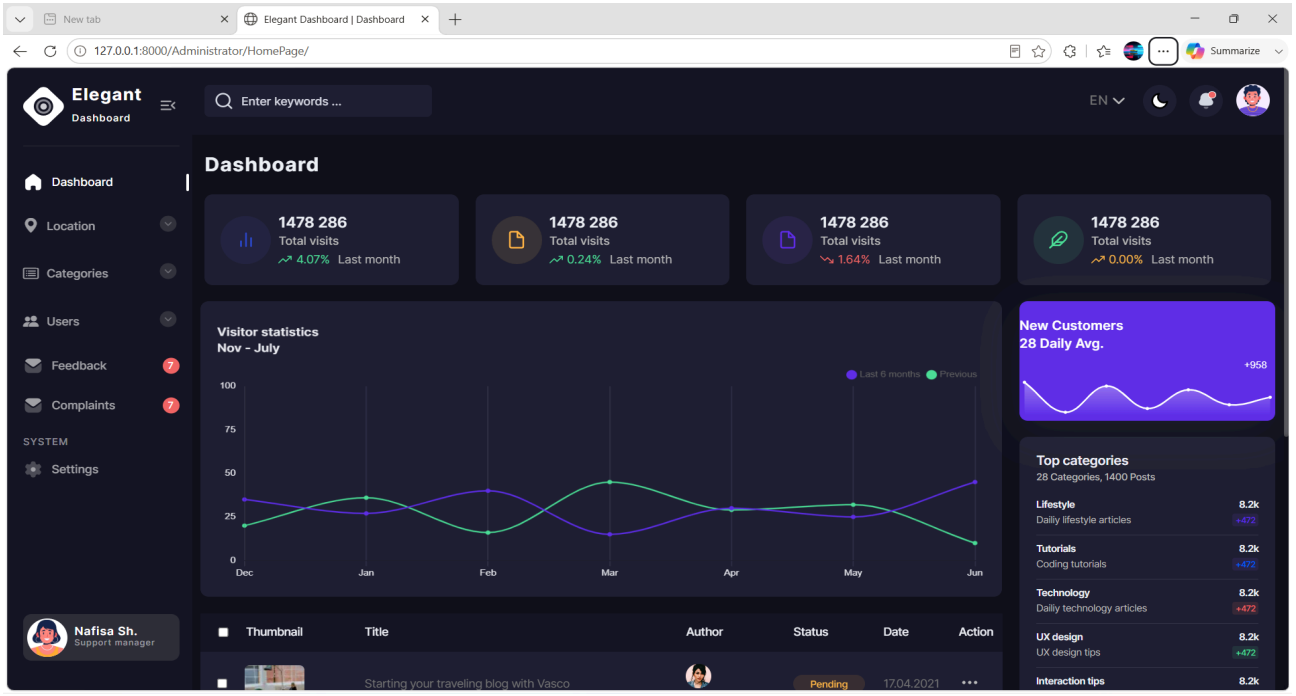
9. APPENDIX

9.1 Screenshort

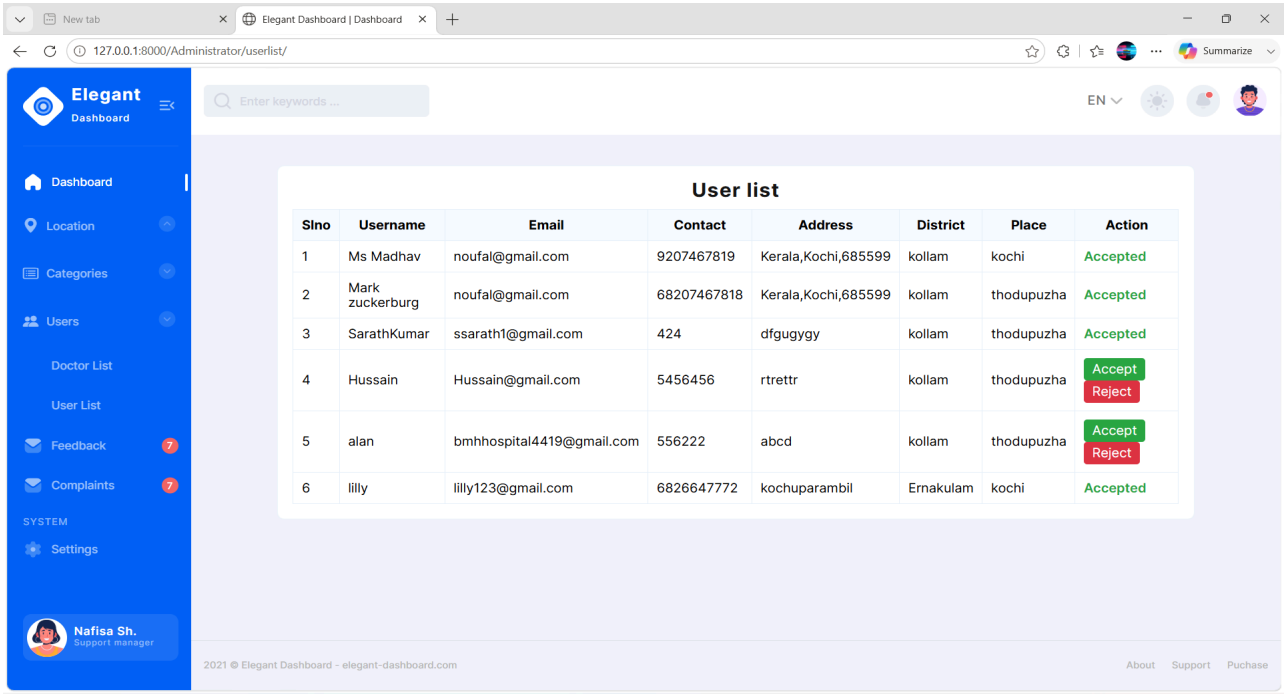
Login Page:



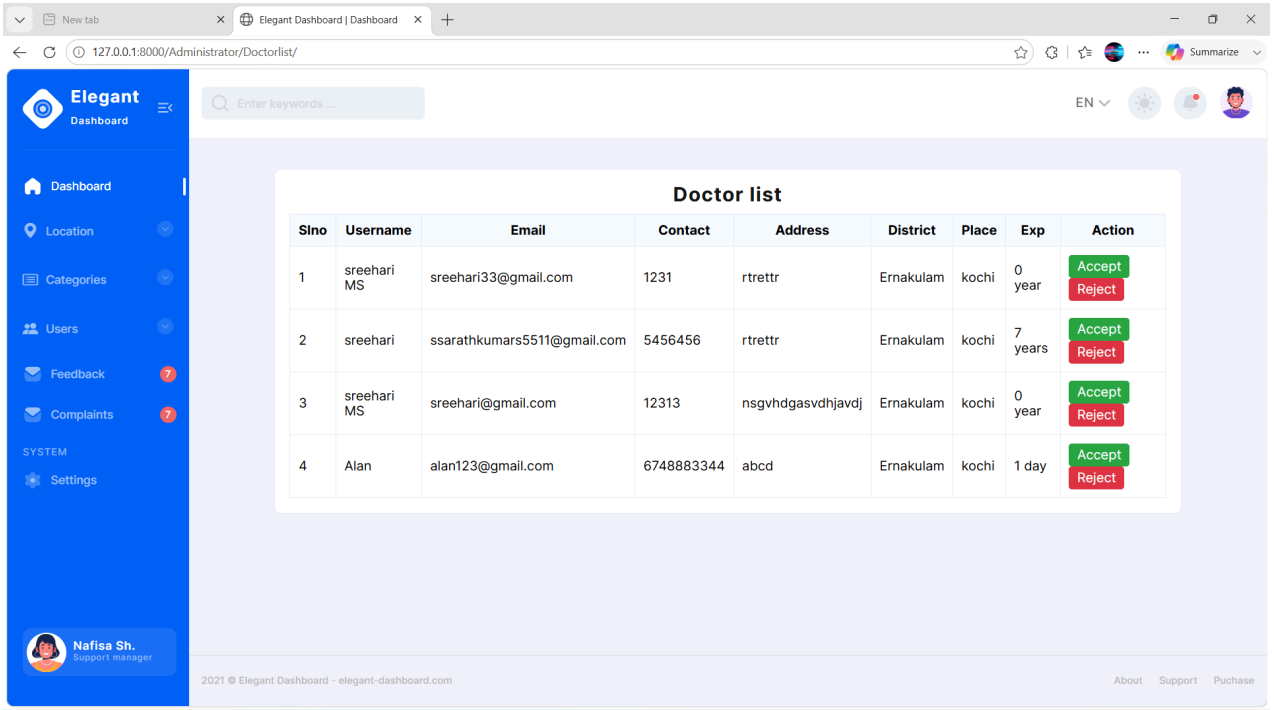
Home Page:



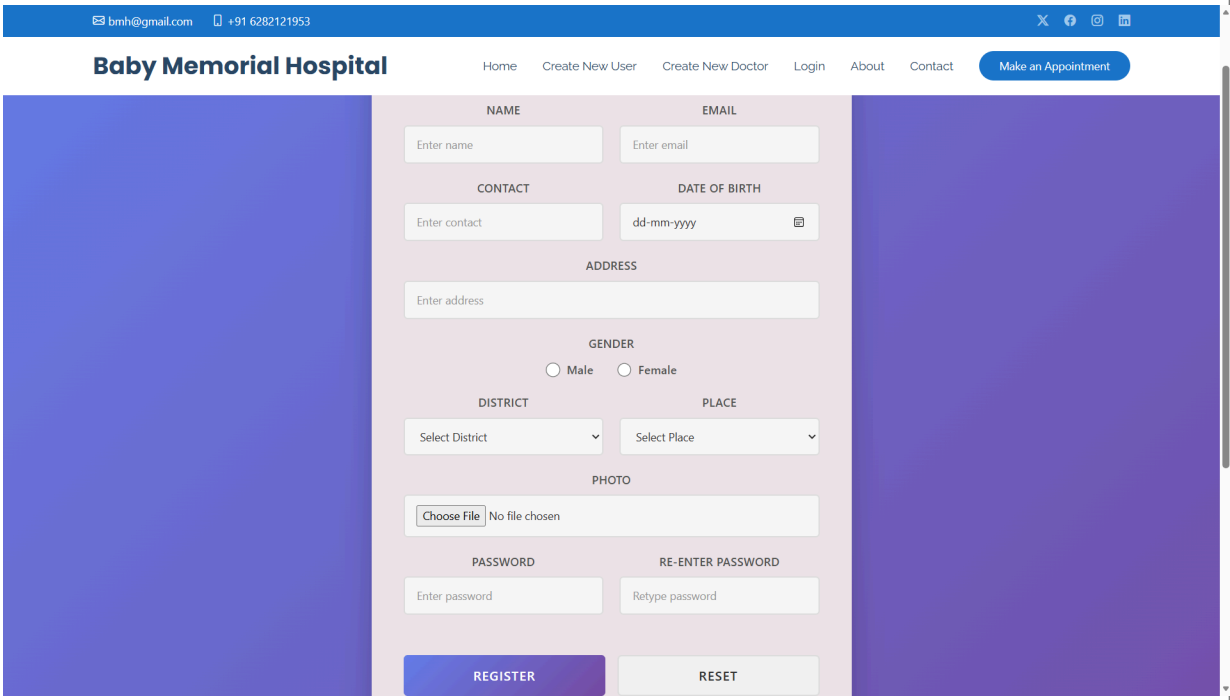
User List:



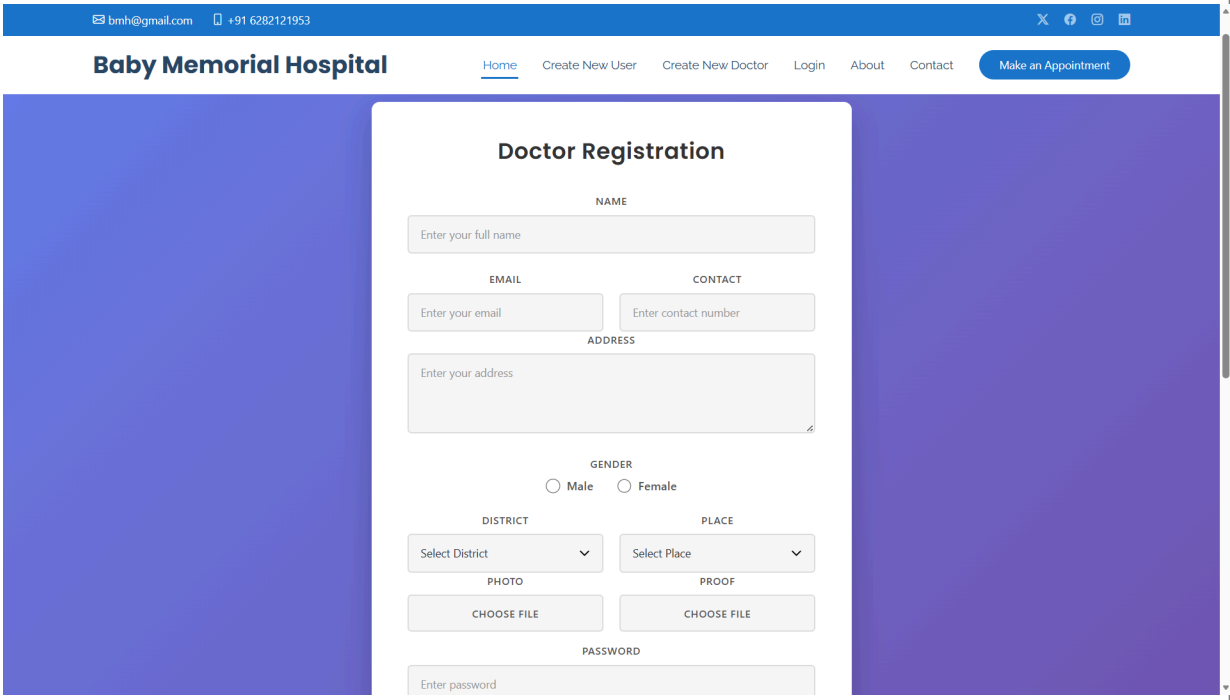
Doctor List:



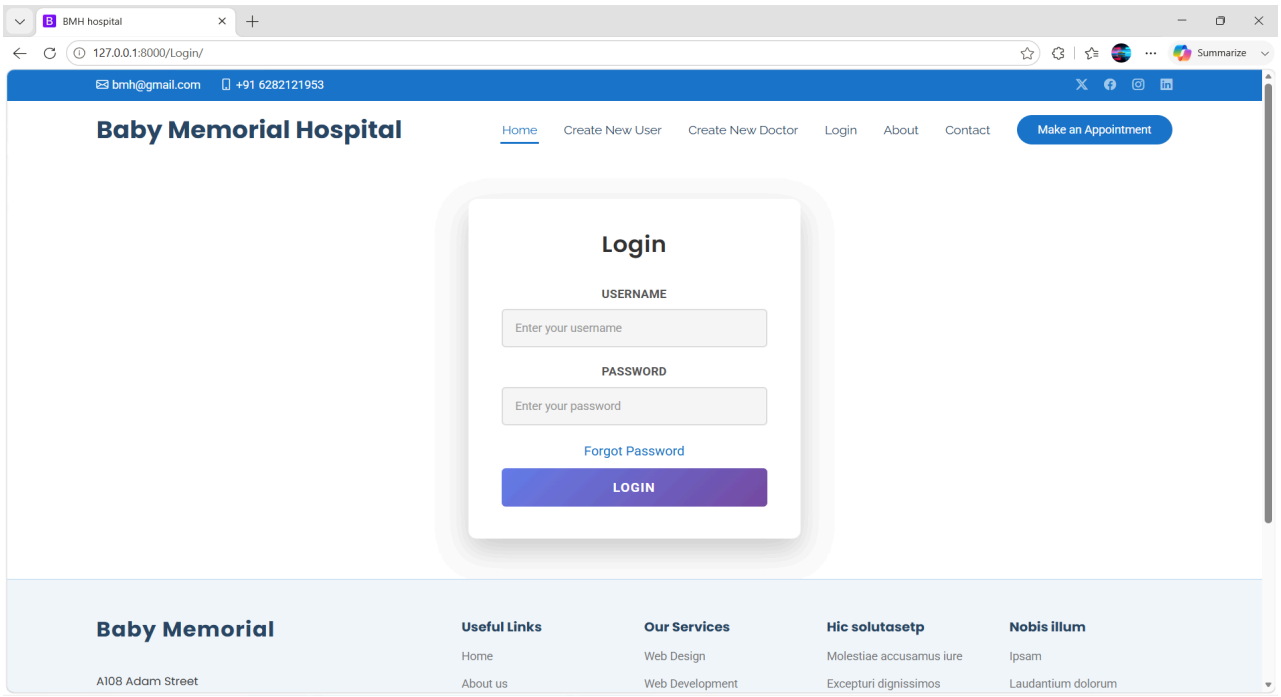
Doctor Registration Form:



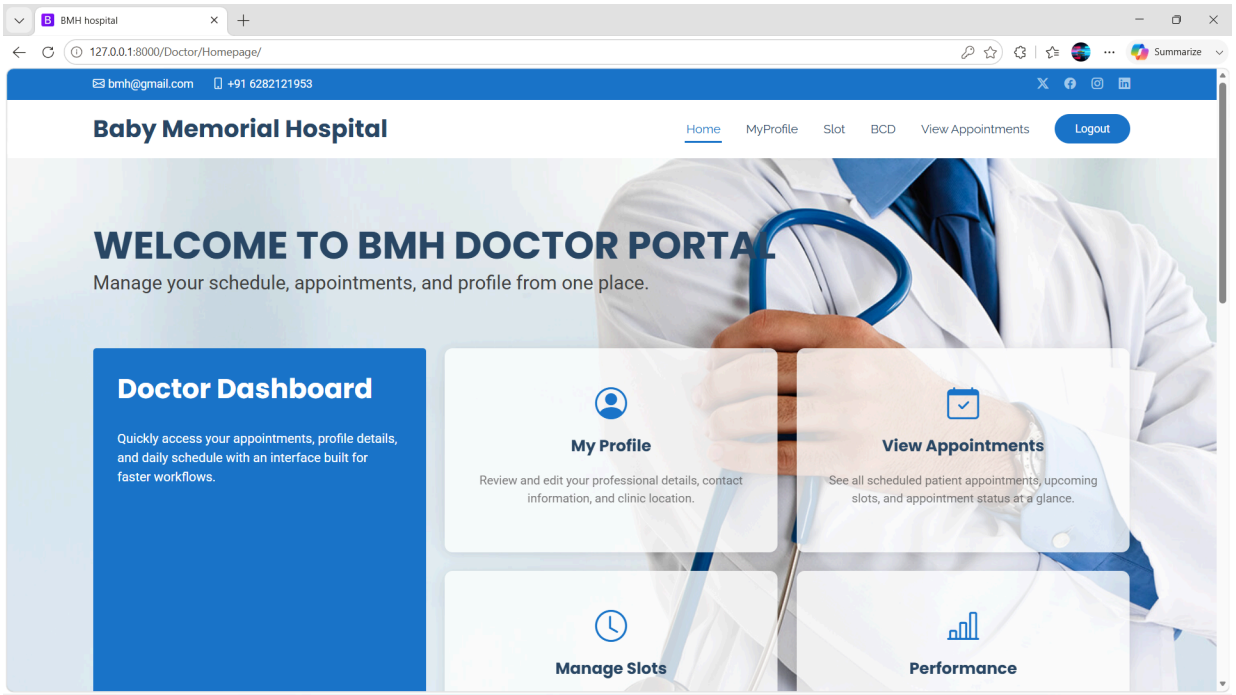
User Registration Form:



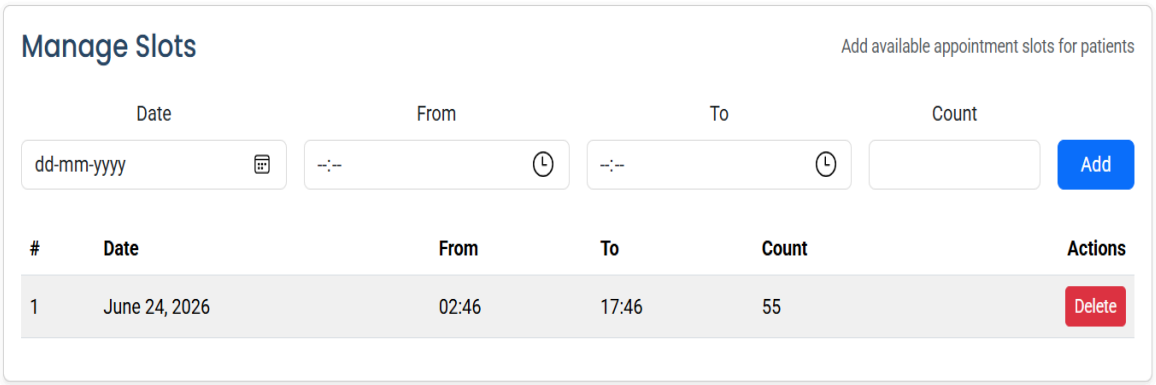
Login:



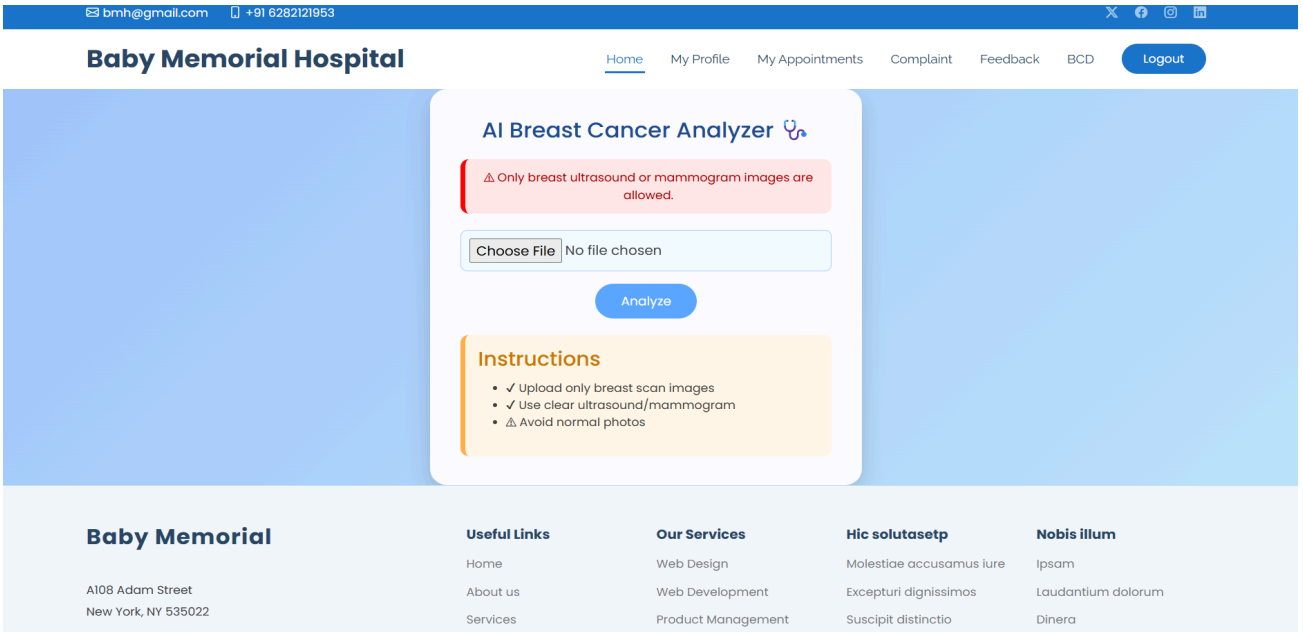
Doctor Homepage:



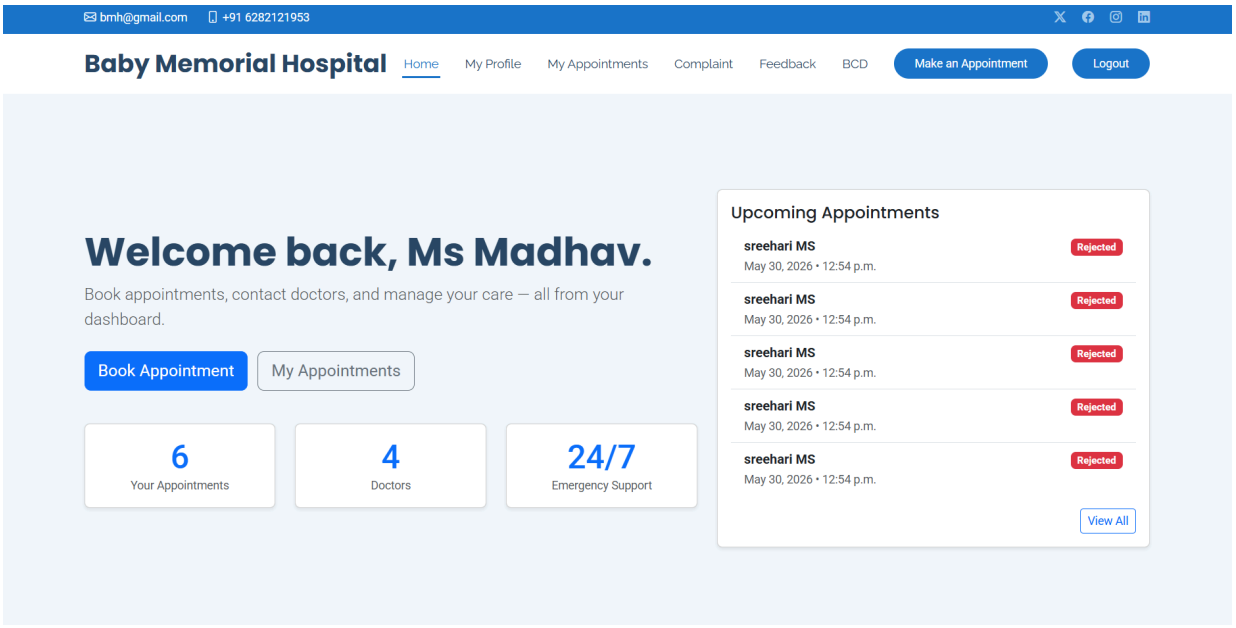
Slot:



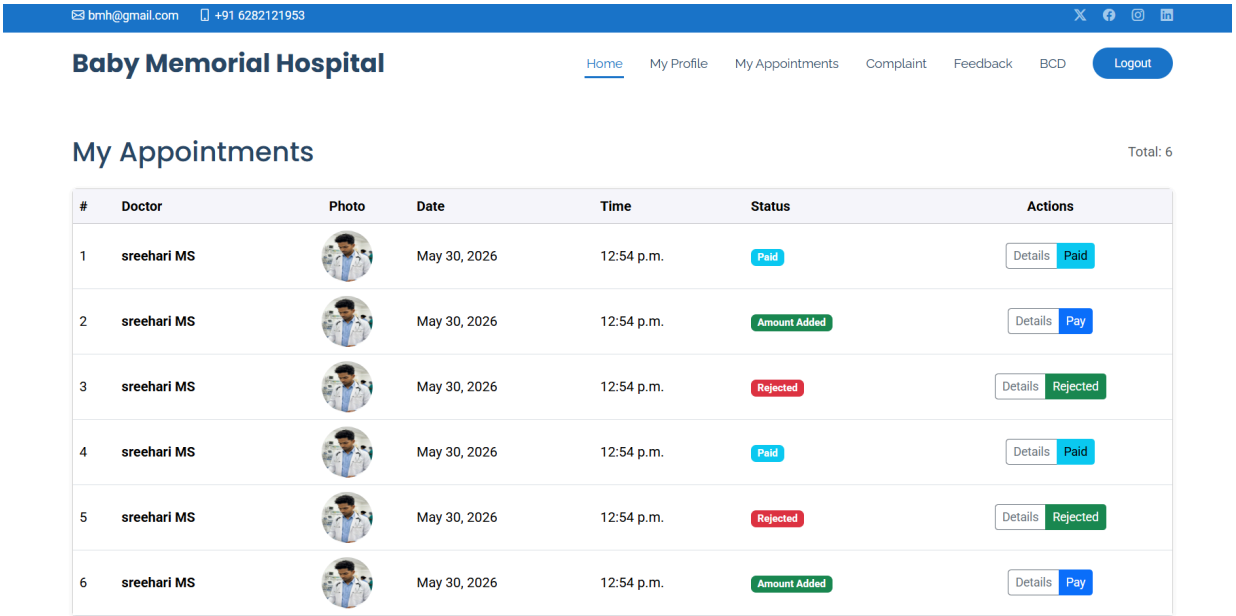
Breast Cancer Detection:



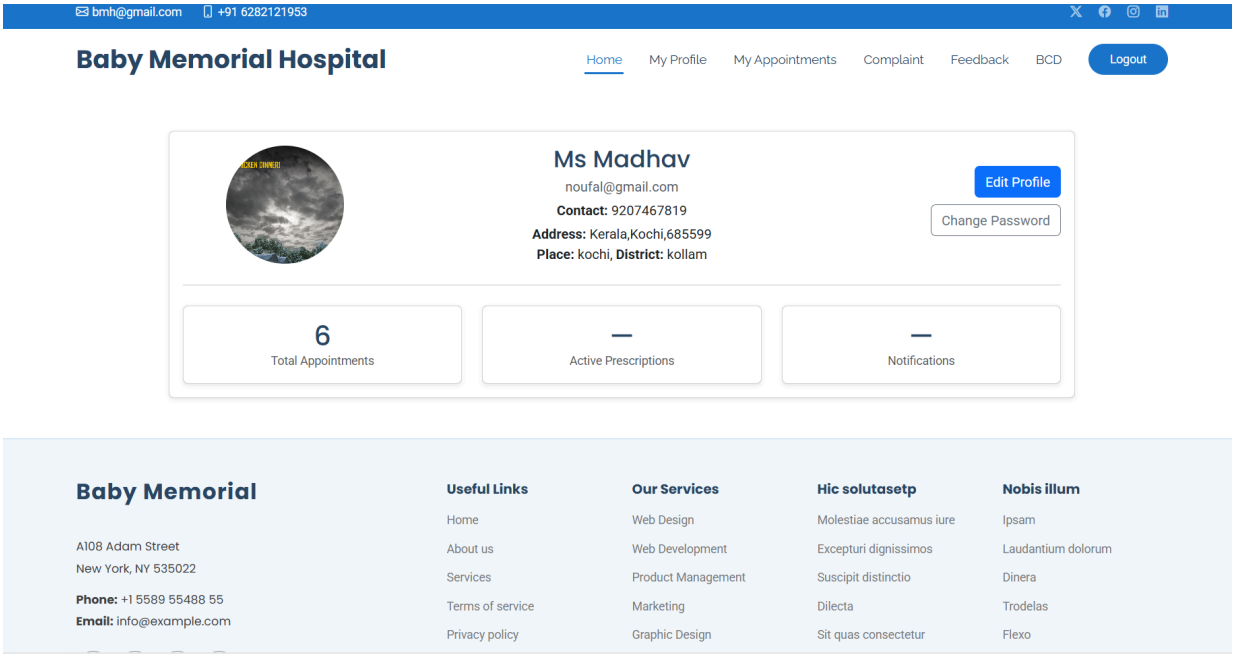
User Homepage:



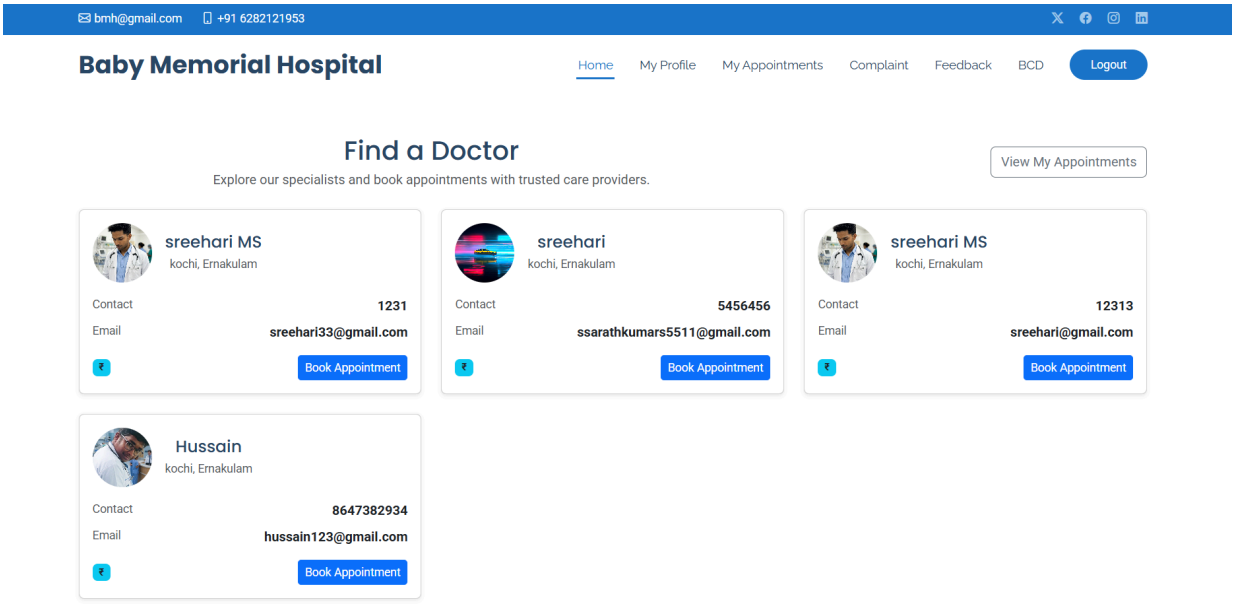
My Appointment:



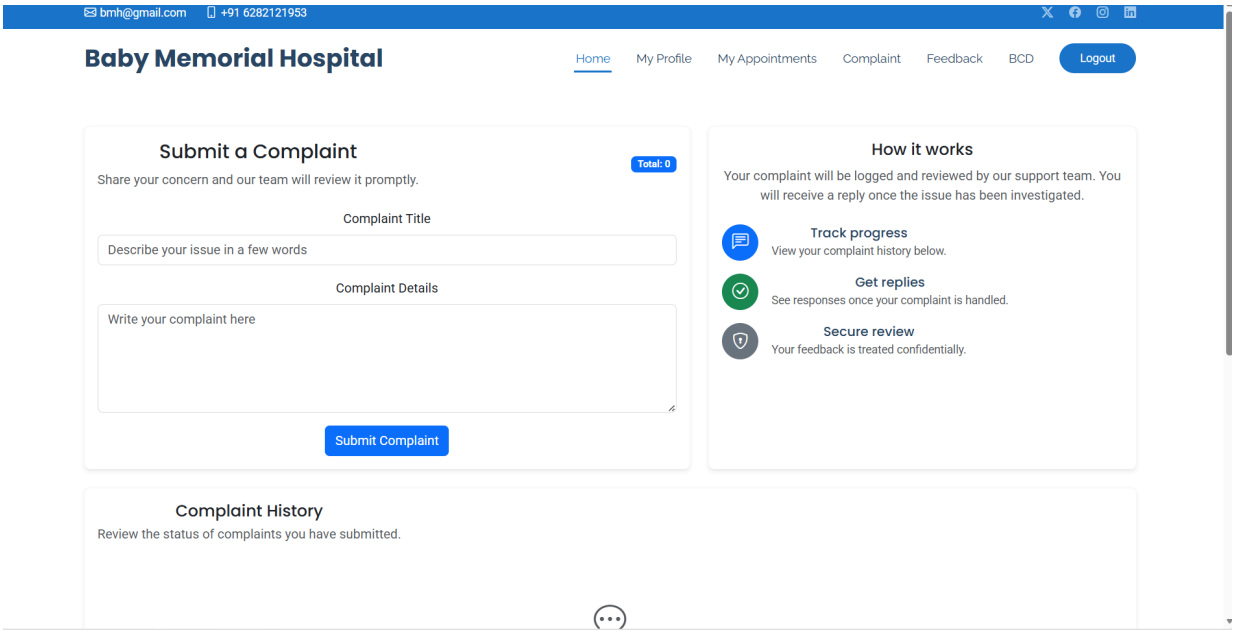
User MyProfile:



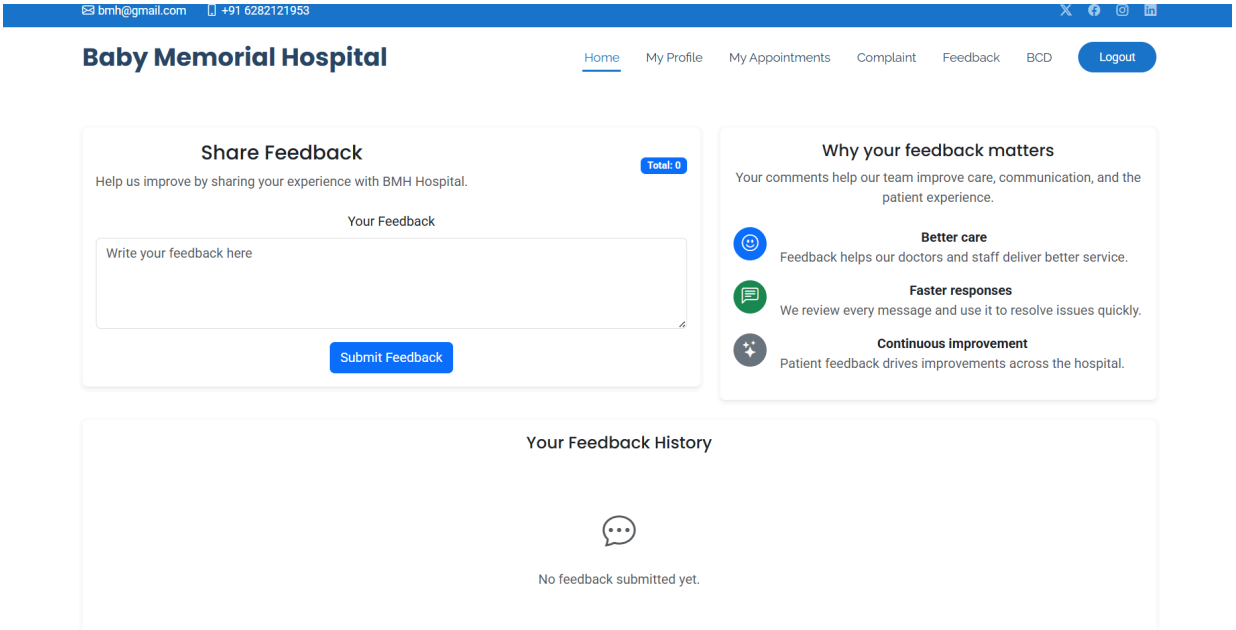
Make An Appointment:



Complaint:



Feedback:



9.2 CODING

Login:

```
{% extends 'Guest/Header.html' %}
{% block content %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }



        .login-container {
            background: white;
            border-radius: 10px;
            box-shadow: 0 15px 35px rgba(0, 0, 0, 0.2);
            width: 100%;
            max-width: 400px;
            padding: 40px;
            margin-top: 50px;
            margin-bottom: 50px;
            animation: slideIn 0.5s ease-out;
        }

        @keyframes slideIn {
            from {
                opacity: 0;
                transform: translateY(-20px);
            }
            to {
                opacity: 1;
                transform: translateY(0);
            }
        }

        .login-header {
            text-align: center;
            margin-bottom: 35px;
        }

        .login-header h2 {
            color: #333;
```

```
    font-size: 28px;
    font-weight: 600;
    letter-spacing: 0.5px;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    color: #555;
    font-size: 14px;
    font-weight: 600;
    margin-bottom: 8px;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.form-group input {
    width: 100%;
    padding: 12px 15px;
    border: 2px solid #e0e0e0;
    border-radius: 5px;
    font-size: 14px;
    color: #333;
    transition: all 0.3s ease;
    background-color: #f9f9f9;
}

.form-group input:focus {
    outline: none;
    border-color: #667eea;
    background-color: #fff;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

.form-group input::placeholder {
    color: #999;
}

.login-button {
    width: 100%;
    padding: 12px 15px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    border: none;
```

```
            border-radius: 5px;
            font-size: 16px;
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 1px;
            cursor: pointer;
            transition: all 0.3s ease;
            margin-top: 10px;
        }

        .login-button:hover {
            transform: translateY(-2px);
            box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
        }

        .login-button:active {
            transform: translateY(0);
        }

        @media (max-width: 480px) {
            .login-container {
                padding: 30px 20px;
            }

            .login-header h2 {
                font-size: 24px;
            }
        }
    </style>
</head>
<body>
    <div class="login-container">
        <div class="login-header">
            <h2>Login</h2>
        </div>

        <form method="post">
            {% csrf_token %}

            <div class="form-group">
                <label for="username">Username</label>
                <input
                    type="text"
                    id="username"
                    name="username"
                    required
                    placeholder="Enter your username"
```

```
        </div>

        <div class="form-group">
          <label for="password">Password</label>
          <input
            type="password"
            id="password"
            name="password"
            required
            placeholder="Enter your password"

          >
        </div>
         <tr>
          <td colspan="2" align="right"><a href="{% url 'Guest:forgotpassword'
%}">Forgot Password</a></td>
        </tr>
        <button type="submit" class="login-button">Login</button>
      </form>
    </div>

    {% if msg %}
    <script>
      alert("{{msg}}");
      window.location="{% url 'Guest:Login' %}"
    </script>
    {% endif %}
</body>
</html>

{% endblock content %}
```

Views:

```
from django.shortcuts import render,redirect
from Guest.models import *
from Administrator.models import *
import random
from django.core.mail import send_mail
from django.conf import settings


# Create your views here.
def login(request):
    if request.method=="POST":
        email = request.POST.get("username")
        password=request.POST.get("password")
```

```
usercount=tbl_userregistration.objects.filter(userregistration_email=email,userregistration_
password=password).count()
        admincount =
tbl_adminregistration.objects.filter(adminregistration_email=email,adminregistration_pass
word=password).count()

doctorcount=tbl_doctorregistration.objects.filter(doctorregistration_email=email,doctorregi
stration_password=password).count()
        if usercount > 0:
            userdata =
tbl_userregistration.objects.get(userregistration_email=email,userregistration_password=p
assword)
            if userdata.user_status==1:
                request.session['uid']=userdata.id
            elif userdata.user_status==2:
                return render(request,"Guest/Login.html",{'msg':"VERIFICATION
REJECTED"})
            elif userdata.user_status==0:
                return render(request,"Guest/Login.html",{'msg':"Verification pending"})
            request.session['uid'] = userdata.id
            return redirect("User:Homepage")

        elif admincount > 0:
            admindata =
tbl_adminregistration.objects.get(adminregistration_email=email,adminregistration_passw
ord=password)
            request.session['aid'] = admindata.id
            return redirect("Administrator:HomePage")

        elif doctorcount > 0:
            doctordata =
tbl_doctorregistration.objects.get(doctorregistration_email=email,doctorregistration_passw
ord=password)
            if doctordata.doctor_status==1:
                request.session['did']=doctordata.id
                return redirect("Doctor:Homepage")
            elif doctordata.doctor_status==2:
                return render(request,"Guest/Login.html",{'msg':"VERIFICATION
REJECTED"})
            elif doctordata.doctor_status==0:
                return render(request,"Guest/Login.html",{'msg':"Verification pending"})
        else:
            return render(request,"Guest/Login.html",{'msg':"Invalid Email Or Password"})
    else:
        return render(request, 'Guest/Login.html')
```

```
def userregistration(request):
        userregistration_data=tbl_userregistration.objects.all()
        district_data=tbl_district.objects.all()



        place_data=tbl_place.objects.all()
        if request.method=="POST":
            name=request.POST.get('txt_name')
            email=request.POST.get('txt_email')
            contact=request.POST.get('txt_contact')
            address=request.POST.get('txt_address')
            gender=request.POST.get('gender')
            dateofbirth=request.POST.get('txt_dob')
            district_id=request.POST.get("select_district")
            district_obj=tbl_district.objects.get(id=district_id)
            place_id=request.POST.get("place_name")
            place_obj=tbl_place.objects.get(id=place_id)
            photo=request.FILES.get('txt_photo')
            password=request.POST.get('txt_pass')

tbl_userregistration.objects.create(userregistration_name=name,userregistration_email=em
ail,userregistration_contact=contact,

userregistration_address=address,userregistration_gender=gender,userregistration_dateofbi
rth=dateofbirth,

userregistration_place=place_obj,userregistration_photo=photo,userregistration_password
=password)
            return redirect("Guest:UserRegistration")
        else:
            return
render(request,'Guest/UserRegistration.html',{'userregistration':userregistration_data,'distri
ct':district_data,'place':place_data})

def ajaxplace(request):
    did=request.GET.get('did')
    place=tbl_place.objects.filter(district=did)
    return render(request,'Guest/Ajaxplace.html',{'place':place})

def doctorregistration(request):
   doctorregistration_data=tbl_doctorregistration.objects.all()
   district_data=tbl_district.objects.all()
   place_data=tbl_place.objects.all()
   if request.method=="POST":
       name=request.POST.get('txt_name')
       email=request.POST.get('txt_email')
       contact=request.POST.get('txt_contact')
       address=request.POST.get('txt_address')
```

```
gender=request.POST.get('gender')
        district_id=request.POST.get("select_district")
        district_obj=tbl_district.objects.get(id=district_id)
        place_id=request.POST.get("place_name")
        place_obj=tbl_place.objects.get(id=place_id)
        photo=request.FILES.get('txt_photo1')
        proof=request.FILES.get('txt_photo2')
        password=request.POST.get('txt_pass')
        exp=request.POST.get('txt_exp')



tbl_doctorregistration.objects.create(doctorregistration_name=name,doctorregistration_em
ail=email,doctorregistration_contact=contact,

doctorregistration_address=address,doctorregistration_gender=gender,doctorregistration_pl
ace=place_obj,

doctorregistration_photo=photo,doctorregistration_password=password,doctorregistration_
proof=proof,
                                    doctorregistration_exp=exp)
        return redirect("Guest:DoctorRegistration")
    else:
        return
render(request,'Guest/DoctorRegistration.html',{'doctorregistration':doctorregistration_data
,'district':district_data,'place':place_data})

def index(request):
    return render(request,'Guest/index.html')

def forgotpassword(request):
    if request.method == "POST":
        email = request.POST.get("txt_email")
        user = tbl_userregistration.objects.get(userregistration_email=email)
        otp = random.randint(111111,999999)
        request.session["otp"] = otp
        request.session["fid"] = user.id
        send_mail(
            'Forgot password OTP', #subject
            "\rHello \r" + str(otp) +"\n This is the OTP to reset ur password.\n If you didn't ask
to reset your password, you can ignore this email. \r\n Thanks. \r\n Your D MARKET
team.",#body
            settings.EMAIL_HOST_USER,
            [email],
        )
        return render(request,"Guest/ForgotPassword.html",{"msg":email})
    else:
        return render(request,"Guest/ForgotPassword.html")
```

```
def otp(request):
    if request.method == "POST":
        inp_otp = int(request.POST.get("txt_otp"))
        if inp_otp == request.session["otp"]:
            return redirect("Guest:newpass")
        else:
            return render(request,"Guest/OTP.html",{"msg":"OTP Does not Matches..!!"})
    else:
        return render(request,"Guest/OTP.html")

def newpass(request):
    if request.method == "POST":
        user = tbl_userregistration.objects.get(id=request.session["fid"])
        if request.POST.get("txt_new_pass") == request.POST.get("txt_con_pass"):



user.user_password = request.POST.get("txt_con_pass")
            user.save()
            return render(request,"Guest/NewPassword.html",{"msg1":"Password Updated
Sucessfully...."})
        else:
            return render(request,"Guest/NewPassword.html",{"msg":"Error in confirm
password..!!!"})
    else:
        return render(request,"Guest/NewPassword.html")
```

User Register

```
{% extends 'Guest/Header.html' %}
{% block content %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>User Registration</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            /* padding: 40px 20px; */
        }
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        .registration-container {
            background: rgb(235, 229, 232);
            border-radius: 10px;
            box-shadow: 0 15px 35px rgba(0, 0, 0, 0.2);
            width: 100%;
```

```
    max-width: 600px;
        padding: 40px;
        animation: slideIn 0.5s ease-out;
    }

    @keyframes slideIn {
        from {
            opacity: 0;
            transform: translateY(-20px);
        }
        to {
            opacity: 1;
            transform: translateY(0);
        }
    }

    .registration-header {
        text-align: center;
        margin-bottom: 35px;
    }

    .registration-header h2 {
        color: #333;
        font-size: 28px;
        font-weight: 600;
        letter-spacing: 0.5px;
```

```
}

    .form-group {
        margin-bottom: 20px;
    }

    .form-row {
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 20px;
    }

    .form-group label {
        display: block;
        color: #555;
        font-size: 14px;
        font-weight: 600;
        margin-bottom: 8px;
        text-transform: uppercase;
        letter-spacing: 0.5px;
    }

    .form-group input,
    .form-group select {
        width: 100%;
        padding: 12px 15px;
```

```
    border: 2px solid #e0e0e0;
            border-radius: 5px;
            font-size: 14px;
            color: #333;
            transition: all 0.3s ease;
            background-color: #f9f9f9;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }

        .form-group input:focus,
        .form-group select:focus {
            outline: none;
            border-color: #667eea;
            background-color: #fff;
            box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
        }

        .form-group input::placeholder {
            color: #999;
        }

        .form-group-full {
            grid-column: 1 / -1;
        }

        .form-group-radio {
```

```
    margin-bottom: 20px;
        }

        .form-group-radio label {
            display: block;
            color: #555;
            font-size: 14px;
            font-weight: 600;
            margin-bottom: 12px;
            text-transform: uppercase;
            letter-spacing: 0.5px;
        }

        .radio-group {
            display: flex;
            gap: 30px;
        }

        .radio-item {
            display: flex;
            align-items: center;
            gap: 8px;
        }

        .radio-item input[type="radio"] {
            width: 18px;
```

```
height: 18px;
            cursor: pointer;
            accent-color: #667eea;
        }

        .radio-item label {
            margin: 0;
            text-transform: none;
            font-weight: 500;
            cursor: pointer;
            font-size: 14px;
        }

        .button-group {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 15px;
            margin-top: 30px;
        }

        .btn-register,
        .btn-reset {
            padding: 12px 15px;
            border: none;
            border-radius: 5px;
            font-size: 16px;
```

```
font-weight: 600;
        text-transform: uppercase;
        letter-spacing: 1px;
        cursor: pointer;
        transition: all 0.3s ease;
    }

    .btn-register {
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        color: white;
    }

    .btn-register:hover {
        transform: translateY(-2px);
        box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
    }

    .btn-register:active {
        transform: translateY(0);
    }

    .btn-reset {
        background-color: #f0f0f0;
        color: #333;
        border: 2px solid #e0e0e0;
    }
```

```
.btn-reset:hover {
        background-color: #e8e8e8;
        transform: translateY(-2px);
    }

    .btn-reset:active {
        transform: translateY(0);
    }

    @media (max-width: 680px) {
        .registration-container {
            padding: 30px 20px;
        }

        .registration-header h2 {
            font-size: 24px;
        }

        .form-row {
            grid-template-columns: 1fr;
        }

        .button-group {
            grid-template-columns: 1fr;
        }
```

```
.radio-group {
            flex-direction: column;
            gap: 12px;
        }
    }
  </style>
</head>

<body>
  <div class="registration-container">
    <div class="registration-header">
      <h2>User Registration</h2>
    </div>

    <form method="post" enctype="multipart/form-data">
      {% csrf_token %}

      <div class="form-row">
        <div class="form-group">
          <label for="username">Name</label>
          <input
            type="text"
            id="username"
            name="txt_name"
            required
```

```
placeholder="Enter name"

            >
          </div>

          <div class="form-group">
            <label for="email">Email</label>
            <input
              type="email"
              id="email"
              name="txt_email"
              required
              placeholder="Enter email"
            >
          </div>
        </div>

        <div class="form-row">
          <div class="form-group">
            <label for="contact">Contact</label>
            <input
              type="text"
              id="contact"
              name="txt_contact"
              required
              placeholder="Enter contact"
```

```
>
            </div>

            <div class="form-group">
               <label for="dob">Date of Birth</label>
               <input
                  type="date"
                  id="dob"
                  name="txt_dob"
                  required
               >
            </div>
         </div>

         <div class="form-group form-group-full">
            <label for="address">Address</label>
            <input
               type="text"
               id="address"
               name="txt_address"
               required
               placeholder="Enter address"
            >
         </div>

         <div class="form-group-radio" style="display: flex; flex-direction: column;">
```

```
<label>Gender</label>
            <div class="radio-group" style="display: flex; flex-direction: row;margin:auto;">
              <div class="radio-item">
                <input
                  type="radio"
                  id="male"
                  name="gender"
                  value="Male"
                  required
                >
                <label for="male">Male</label>
              </div>
              <div class="radio-item">
                <input
                  type="radio"
                  id="female"
                  name="gender"
                  value="Female"
                  required
                >
                <label for="female">Female</label>
              </div>
            </div>
          </div>


          <div class="form-row">
```

```
<div class="form-group">
            <label for="sel_district">District</label>
            <select name="select_district" id="sel_district" required>
              <option value="">Select District</option>
              {% for i in district %}
              <option value="{{i.id}}">{{i.district_name}}</option>
              {% endfor %}
            </select>
          </div>

          <div class="form-group">
            <label for="sel_place">Place</label>
            <select name="place_name" id="sel_place" required>
              <option value="">Select Place</option>
            </select>
          </div>
        </div>

        <div class="form-group form-group-full">
          <label for="txt_photo">Photo</label>
          <input
            type="file"
            id="txt_photo"
            name="txt_photo"
            required
```

```
>
        </div>

         <div class="form-row">
            <div class="form-group">
               <label for="password">Password</label>
               <input
                  type="password"
                  id="password"
                  name="txt_pass"
                  required
                  placeholder="Enter password"
                  pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"
                  title="Must contain at least one number, one uppercase and lowercase letter, and at least 8 or more characters"
               >
            </div>

            <div class="form-group">
               <label for="password2">Re-enter Password</label>
               <input
                  type="password"
                  id="password2"
                  name="txt_pass1"
                  required
```

```
placeholder="Retype password>
                </div>
            </div>

            <div class="button-group">
                <button type="submit" class="btn-register">Register</button>
                <button type="reset" class="btn-reset">Reset</button>
            </div>
        </form>
    </div>

</body>
</html>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
<script>
    $('#sel_district').change(function () {
        var did = $(this).val();
        var ur = "{% url 'Guest:ajaxplace' %}"
        $.ajax({
            url: ur,
            data: { did: did },
            success: function (data) {
                $('#sel_place').html(data);
            }
        });
    });
```

```
</script>

{% endblock content %}
```

Views:

```
def userregistration(request):
        userregistration_data=tbl_userregistration.objects.all()
        district_data=tbl_district.objects.all()
        place_data=tbl_place.objects.all()
        if request.method=="POST":
            name=request.POST.get('txt_name')
            email=request.POST.get('txt_email')
            contact=request.POST.get('txt_contact')
            address=request.POST.get('txt_address')
            gender=request.POST.get('gender')
            dateofbirth=request.POST.get('txt_dob')
            district_id=request.POST.get("select_district")
            district_obj=tbl_district.objects.get(id=district_id)
            place_id=request.POST.get("place_name")
            place_obj=tbl_place.objects.get(id=place_id)
            photo=request.FILES.get('txt_photo')
            password=request.POST.get('txt_pass')

tbl_userregistration.objects.create(userregistration_name=name,userregistration_email=em
ail,userregistration_contact=contact,

userregistration_address=address,userregistration_gender=gender,userregistration_dateofbi
rth=dateofbirth,

userregistration_place=place_obj,userregistration_photo=photo,userregistration_password
=password)
            return redirect("Guest:UserRegistration")
        else:
            return
render(request,'Guest/UserRegistration.html',{'userregistration':userregistration_data,'distri
ct':district_data,'place':place_data})
```

Report:

```
{% extends 'User/Header.html' %}
{% load static %}
{% block content %}
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
```

```
<form action="" method="POST" class="container mt-5">
{% csrf_token %}
<div class="row justify-content-center">
<div class="col-md-8">
<div class="card shadow mb-4">
<div class="card-body">
<h2 class="text-center mb-4">Submit Report</h2>
<div class="row mb-3">
<label for="txtcontent" class="col-md-3 col-form-label">Content</label>
<div class="col-md-9"
```